

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ

к лабораторной работе №1

на тему:

**«СИММЕТРИЧНАЯ КРИПТОГРАФИЯ. СТАНДАРТ ШИФРОВАНИЯ
ГОСТ 28147-89»**

БГУИР 6-05-0612-02 73

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

Проверил ассистент каф. Информатики
ПОЛЯКОВ Кирилл Борисович

Минск 2026

СОДЕРЖАНИЕ

Введение.....	3
1 Выполнение работы	5
Вывод.....	8
Приложение А (обязательное) Исходный код программы	9

ВВЕДЕНИЕ

ГОСТ 28147-89 «Системы обработки информации. Защита криптографическая. Алгоритм криптографического преобразования» – устаревший государственный стандарт СССР (позже межгосударственный стандарт СНГ), описывающий алгоритм симметричного блочного шифрования и режимы его работы.

Является примером DES-подобных криптосистем, созданных по классической итерационной схеме Фейстеля.

ГОСТ 28147-89 представляет собой симметричный 64-битовый блочный алгоритм с 256-битовым ключом.

Этот алгоритм криптографического преобразования данных предназначен для аппаратной и программной реализации, удовлетворяет криптографическим требованиям и до 1983 года не накладывал ограничений на степень секретности защищаемой информации.

Схема работы алгоритма ГОСТ 28147-89 следующая. Данные, подлежащие зашифровке, разбивают на 64-разрядные блоки. Эти блоки разбиваются на два субблока N1 и N2 по 32 бит. Шифруемый блок данных разбивается на две части, которые затем обрабатываются как отдельные 32-битовые целые числа без знака. Сначала правая половина блока и подключ раунда складываются по модулю 2^{32} . Затем производится поблочная подстановка.

В итоге 32-битовое значение, полученное на предыдущем шаге интерпретируется как массив из восьми 4-битовых блоков кода. Далее значение каждого из восьми блоков заменяется на новое, которое выбирается по таблице замен.

В каждой строке таблицы замен записаны числа от 0 до 15 в произвольном порядке без повторений. Значения элементов таблицы замен взяты от 0 до 15, так как в четырех битах, которые подвергаются подстановке, может быть записано целое число без знака в диапазоне от 0 до 15. Значение блока заменится на число, стоящее на позиции, номер которой равен значению заменяемого блока.

После выполнения подстановки все 4-битовые блоки снова объединяются в единое 32-битное слово, которое затем циклически сдвигается на 11 битов влево.

Наконец, с помощью побитовой операции XOR результат объединяется с левой половиной, вследствие чего получается новая правая половина R_i .

Новая левая часть L_i берется равной младшей части преобразуемого блока: $L_i = R_{i-1}$.

Полученное значение преобразуемого блока рассматривается как результат выполнения одного раунда алгоритма шифрования.

Алгоритм использует за 32 раунда. Исходный 256-битный ключ разбивается на восемь 32-битных подключей, используемых в той или иной последовательности для шифрования и дешифрования соответственно на протяжении всех раундов.

ГОСТ 28147-89 является блочным шифром, поэтому преобразование данных осуществляется блоками в так называемых базовых циклах.

Базовые циклы заключаются в многократном выполнении для блока данных основного раунда, описанного выше, с использованием разных элементов ключа и отличаются друг от друга порядком использования ключевых элементов.

Существует несколько режимов работы алгоритма.

Режим простой замены, в котором все блоки шифруются независимо друг от друга с разными подключами в разных раундах. Для одинаковых блоков данных блоки шифртекста будут одинаковыми.

Режим гаммирования, в котором в блоки записывается 64-битовая синхропосылка (вектор инициализации) и шифруется с использованием СК. Результат подается на вход регистров и снова шифруется с использованием ключа. В режиме гаммирования с обратной связью для заполнения блоков, начиная со 2-го блока, используется результат зашифрования предыдущего блока открытого текста.

Дополнительно алгоритм позволяет работать в режиме генерации имитоприставок. Исходные данные последовательно подаются на вход алгоритма, где каждый блок обрабатывается с использованием того же ключа шифрования. На выходе получается фиксированный по длине блок данных, называемый имитоприставкой. Имитовставка прикрепляется к зашифрованному сообщению. Получатель после расшифрования вычисляет имитовставку заново из восстановленного открытого текста и сравнивает с полученной, что обеспечивает целостность и аутентичность данных.

Задание:

1 Изучить теоретические сведения.

2 Реализовать программные средства шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в следующих режимах:

- простой замены;
- гаммирования;
- гаммирования с обратной связью;
- генерации имитоприставок.

1 ВЫПОЛНЕНИЕ РАБОТЫ

Для алгоритма шифрования ГОСТ 28147-89 в режиме простой замены сделана блок-схема, представленная на рисунке 1.1.

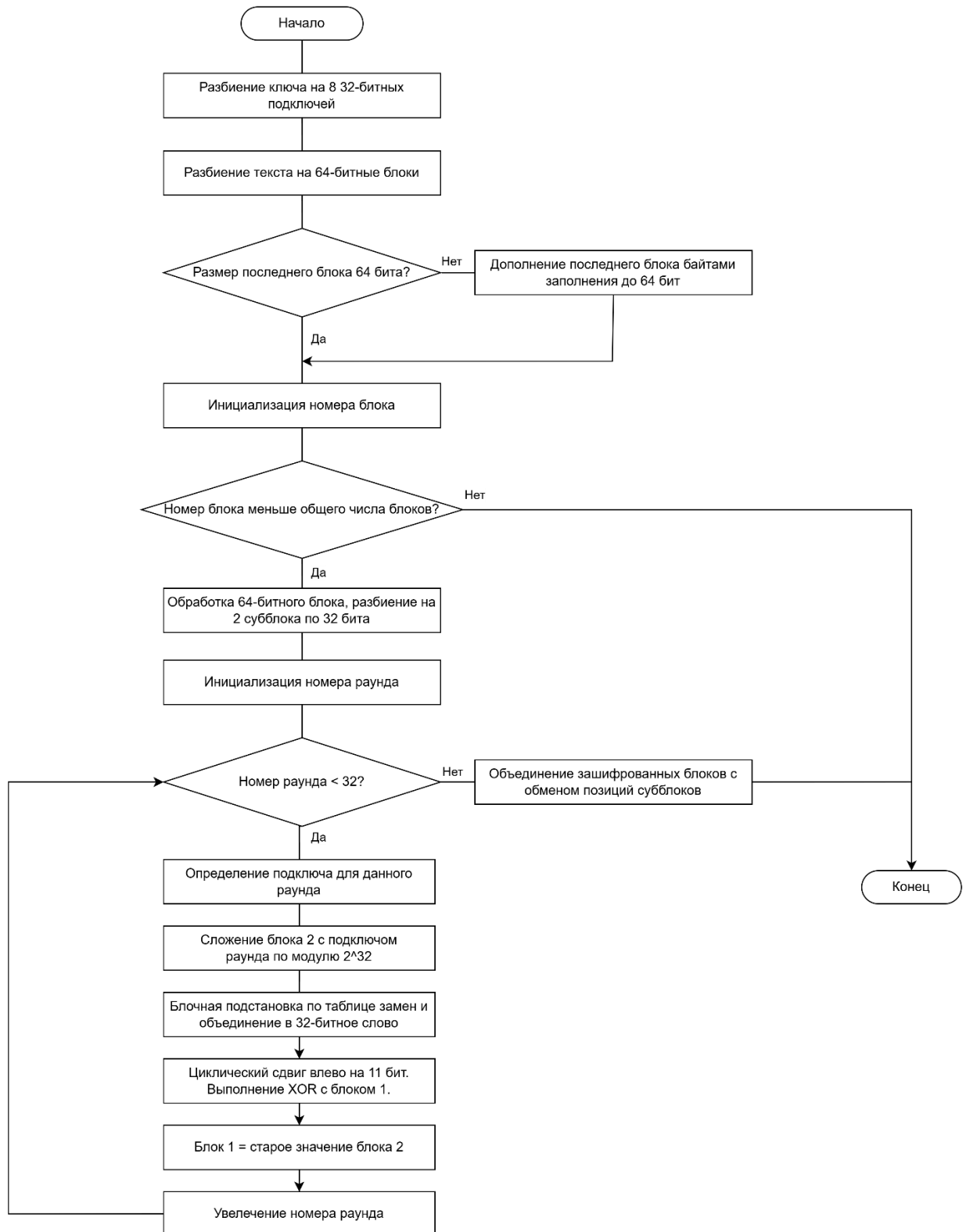


Рисунок 1.1 – Блок-схема алгоритма шифрования ГОСТ 28147-89

Для проверки работы разработанного алгоритма во всех режимах был подготовлен тестовый файл. Его содержимое представлено на рисунке 1.2.

```
test_input.txt
1  GOST 28147-89 test file.
2  Simple replacement, gamma mode, feedback mode, and imitation insert.
3
```

Рисунок 1.2 – Содержимое тестового файла

Сперва программа была запущена в режиме простой замены. Результат выполнения работы программы представлен на рисунке 1.3.

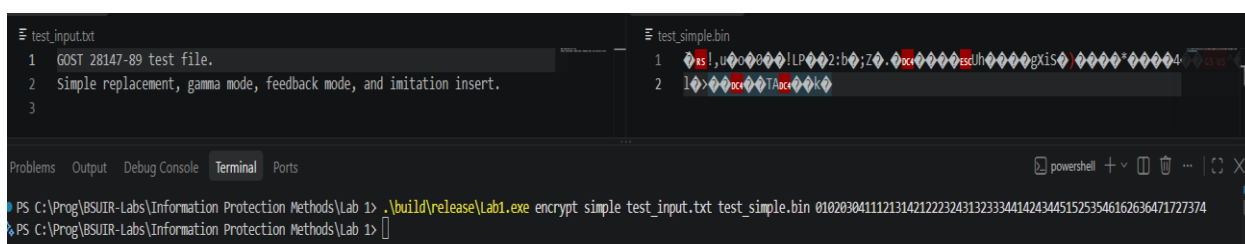


Рисунок 1.3 – Результат выполнения работы программы в режиме простой замены

Далее программа была запущена в режиме гаммирования. Результат выполнения работы программы представлен на рисунке 1.4.



Рисунок 1.4 – Результат выполнения работы программы в режиме гаммирования

После программа была запущена в режиме гаммирования с обратной связью. Результат выполнения работы программы в режиме гаммирования с обратной связью представлен на рисунке 1.5.

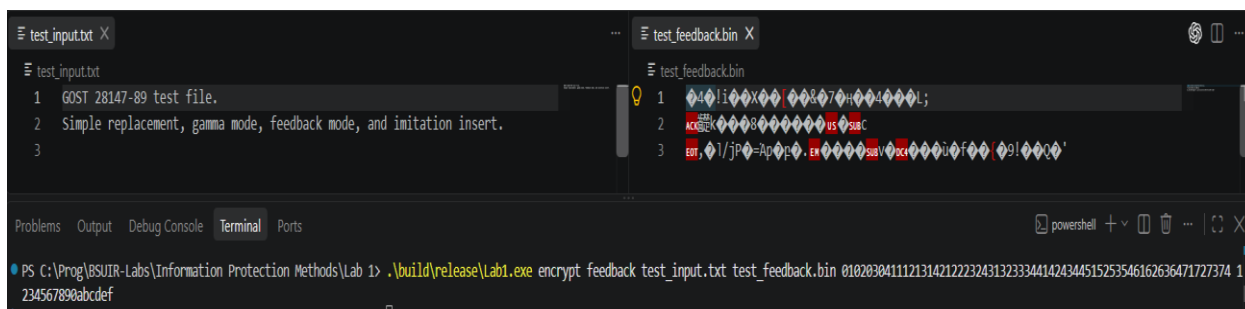


Рисунок 1.5 – Результат выполнения работы программы в режиме гаммирования с обратной связью

Далее программа была запущена в режиме генерации имитоприставки. Результат выполнения работы программы с генерацией имитоприставки представлен на рисунке 1.6.

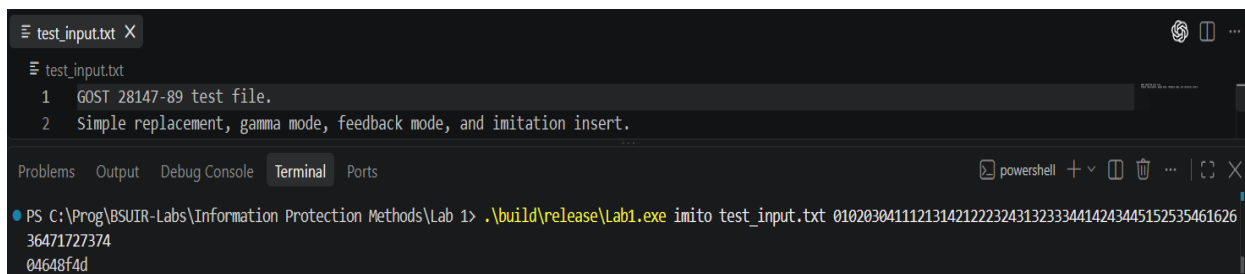


Рисунок 1.6 – Результат выполнения работы программы с генерацией имитоприставки

ВЫВОД

В ходе выполнения лабораторной работы был изучен и программно реализован алгоритм симметричного блочного шифрования ГОСТ 28147-89. Были рассмотрены структура 64-битного блока данных, 256-битного ключа, порядок формирования подключей, раундовое преобразование, таблица нелинейных замен и циклический сдвиг результата на 11 бит.

В рамках работы было разработано программное средство для обработки текстовых файлов в четырёх режимах: простой замены, гаммирования, гаммирования с обратной связью и генерации имитоприставки. Для режима простой замены также была реализована обработка заполнения последнего блока, необходимая для работы с файлами произвольной длины.

Работоспособность программы была проверена на тестовом файле. В режимах простой замены, гаммирования и гаммирования с обратной связью после последующего расшифрования исходное содержимое файла полностью восстанавливается. Для исходного файла была рассчитана имитоприставка, которая может использоваться для контроля целостности сообщения: при изменении содержимого файла результат её вычисления должен измениться.

Таким образом, в ходе лабораторной работы были закреплены практические навыки реализации симметричных криптографических алгоритмов, обработки файлов, применения блочных шифров и использования различных режимов работы стандарта ГОСТ 28147-89.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

```
#include <array>
#include <cstdint>

struct GBlock64
{
    std::uint32_t n1;
    std::uint32_t n2;
};

using GKey = std::array<std::uint32_t, 8>;
using GTable = std::array<std::array<std::uint8_t, 16>, 8>;
using GSubKeys = std::array<std::uint32_t, 32>;

constexpr GTable GSTable = {{
    {{ 4, 10, 9, 2, 13, 8, 0, 14, 6, 11, 1, 12, 7, 15, 5, 3 }},
    {{14, 11, 4, 12, 6, 13, 15, 10, 2, 3, 8, 1, 0, 7, 5, 9 }},
    {{ 5, 8, 1, 13, 10, 3, 4, 2, 14, 15, 12, 7, 6, 0, 9, 11 }},
    {{ 7, 13, 10, 1, 0, 8, 9, 15, 14, 4, 6, 12, 11, 2, 5, 3 }},
    {{ 6, 12, 7, 1, 5, 15, 13, 8, 4, 10, 9, 14, 0, 3, 11, 2 }},
    {{ 4, 11, 10, 0, 7, 2, 1, 13, 3, 6, 8, 5, 9, 12, 15, 14 }},
    {{13, 11, 4, 1, 3, 15, 5, 9, 0, 10, 14, 7, 6, 8, 2, 12 }},
    {{ 1, 15, 13, 0, 5, 7, 10, 4, 9, 2, 3, 14, 6, 11, 8, 12 }}
}};

#include "GStructures.h"

#include <algorithm>
#include <array>
#include <cstdint>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <iterator>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

using Byte = std::uint8_t;
using ByteVector = std::vector<Byte>;

std::uint32_t substitute(std::uint32_t value)
{
    std::uint32_t result = 0;
    for (std::size_t sBoxIndex = 0; sBoxIndex < 8; ++sBoxIndex)
    {
        const std::uint32_t inputValue = (value >> (4 * sBoxIndex)) &
0x0F;
        result |=
static_cast<std::uint32_t>(GSTable[sBoxIndex][inputValue]) << (4 * sBoxIndex);
    }
    return result;
}

std::uint32_t rotateLeft(std::uint32_t value, unsigned shift)
{
    return (value << shift) | (value >> (32 - shift));
}
```

```

        std::uint32_t    roundFunction(std::uint32_t    rightPart,    std::uint32_t
roundKey)
    {
        return rotateLeft(substitute(rightPart + roundKey), 11);
    }

GSubKeys createEncryptionSubKeys(const GKey& key)
{
    GSubKeys subKeys{};
    std::size_t subKeyIndex = 0;
    for (std::size_t repetition = 0; repetition < 3; ++repetition)
    {
        for (std::size_t keyIndex = 0; keyIndex < key.size(); ++keyIndex)
        {
            subKeys[subKeyIndex++] = key[keyIndex];
        }
    }
    for (int keyIndex = static_cast<int>(key.size()) - 1; keyIndex >= 0;
--keyIndex)
    {
        subKeys[subKeyIndex++] = key[static_cast<std::size_t>(keyIndex)];
    }
    return subKeys;
}

GSubKeys createDecryptionSubKeys(const GKey& key)
{
    GSubKeys subKeys = createEncryptionSubKeys(key);
    std::reverse(subKeys.begin(), subKeys.end());
    return subKeys;
}

GBlock64 transformBlock(const GBlock64& block, const GSubKeys& subKeys)
{
    std::uint32_t leftPart = block.n1;
    std::uint32_t rightPart = block.n2;
    for (std::size_t round = 0; round < 31; ++round)
    {
        const std::uint32_t nextLeftPart = rightPart;
        const std::uint32_t nextRightPart = leftPart ^
roundFunction(rightPart, subKeys[round]);
        leftPart = nextLeftPart;
        rightPart = nextRightPart;
    }
    leftPart ^= roundFunction(rightPart, subKeys[31]);
    return { leftPart, rightPart };
}

GBlock64 encryptBlock(const GBlock64& block, const GKey& key)
{
    return transformBlock(block, createEncryptionSubKeys(key));
}

GBlock64 decryptBlock(const GBlock64& block, const GKey& key)
{
    return transformBlock(block, createDecryptionSubKeys(key));
}

std::uint32_t readUint32LittleEndian(const Byte* bytes)
{
    return static_cast<std::uint32_t>(bytes[0]) |
        (static_cast<std::uint32_t>(bytes[1]) << 8) |
        (static_cast<std::uint32_t>(bytes[2]) << 16) |

```

```

        (static_cast<std::uint32_t>(bytes[3]) << 24);
    }

    void writeUInt32LittleEndian(Byte* bytes, std::uint32_t value)
    {
        bytes[0] = static_cast<Byte>(value);
        bytes[1] = static_cast<Byte>(value >> 8);
        bytes[2] = static_cast<Byte>(value >> 16);
        bytes[3] = static_cast<Byte>(value >> 24);
    }

    GBlock64 bytesToBlock(const Byte* bytes)
    {
        return { readUInt32LittleEndian(bytes), readUInt32LittleEndian(bytes
+ 4) };
    }

    std::array<Byte, 8> blockToBytes(const GBlock64& block)
    {
        std::array<Byte, 8> bytes{};
        writeUInt32LittleEndian(bytes.data(), block.n1);
        writeUInt32LittleEndian(bytes.data() + 4, block.n2);
        return bytes;
    }

    ByteVector readBinaryFile(const std::string& filePath)
    {
        std::ifstream inputFile(filePath, std::ios::binary);
        if (!inputFile) throw std::runtime_error("Cannot open input file: "
+ filePath);
        return { std::istreambuf_iterator<char>(inputFile),
std::istreambuf_iterator<char>() };
    }

    void writeBinaryFile(const std::string& filePath, const ByteVector& data)
    {
        std::ofstream outputFile(filePath, std::ios::binary);
        if (!outputFile) throw std::runtime_error("Cannot open output file:
" + filePath);
        outputFile.write(reinterpret_cast<const char*>(data.data()),
static_cast<std::streamsize>(data.size()));
    }

    ByteVector addPadding(const ByteVector& data)
    {
        ByteVector paddedData = data;
        const Byte paddingSize = static_cast<Byte>(8 - paddedData.size() %
8);
        paddedData.insert(paddedData.end(), paddingSize, paddingSize);
        return paddedData;
    }

    ByteVector removePadding(const ByteVector& data)
    {
        if (data.empty() || data.size() % 8 != 0) throw
std::runtime_error("Invalid decrypted data length");
        const Byte paddingSize = data.back();
        if (paddingSize == 0 || paddingSize > 8 || paddingSize > data.size())
throw std::runtime_error("Invalid final block padding");
        if (!std::all_of(data.end() - paddingSize, data.end(),
[paddingSize](Byte value) { return value == paddingSize; })) throw
std::runtime_error("Invalid final block padding");
        return ByteVector(data.begin(), data.end() - paddingSize);
    }

```

```

ByteVector encryptBlocks(const ByteVector& data, const GKey& key)
{
    const ByteVector paddedData = addPadding(data);
    ByteVector encryptedData(paddedData.size());
    for (std::size_t offset = 0; offset < paddedData.size(); offset += 8)
    {
        const auto encryptedBlock =
blockToBytes(encryptBlock(bytesToBlock(paddedData.data() + offset), key));
        std::copy(encryptedBlock.begin(), encryptedBlock.end(),
encryptedData.begin() + static_cast<std::ptrdiff_t>(offset));
    }
    return encryptedData;
}

ByteVector decryptBlocks(const ByteVector& data, const GKey& key)
{
    if (data.empty() || data.size() % 8 != 0) throw
std::runtime_error("Ciphertext length must be a multiple of 8 bytes");
    ByteVector decryptedData(data.size());
    for (std::size_t offset = 0; offset < data.size(); offset += 8)
    {
        const auto decryptedBlock =
blockToBytes(decryptBlock(bytesToBlock(data.data() + offset), key));
        std::copy(decryptedBlock.begin(), decryptedBlock.end(),
decryptedData.begin() + static_cast<std::ptrdiff_t>(offset));
    }
    return removePadding(decryptedData);
}

void incrementCounter(GBlock64& counter)
{
    ++counter.n1;
    if (counter.n1 == 0) ++counter.n2;
}

ByteVector gammaCrypt(const ByteVector& data, const GKey& key, GBlock64
counter)
{
    ByteVector result(data.size());
    for (std::size_t offset = 0; offset < data.size(); offset += 8)
    {
        const auto gammaBytes = blockToBytes(encryptBlock(counter, key));
        const std::size_t bytesToProcess = std::min<std::size_t>(8,
data.size() - offset);
        for (std::size_t byteIndex = 0; byteIndex < bytesToProcess;
++byteIndex) result[offset + byteIndex] = data[offset + byteIndex] ^
gammaBytes[byteIndex];
        incrementCounter(counter);
    }
    return result;
}

ByteVector feedbackCrypt(const ByteVector& data, const GKey& key,
GBlock64 feedback, bool decrypting)
{
    ByteVector result(data.size());
    for (std::size_t offset = 0; offset < data.size(); offset += 8)
    {
        const auto gammaBytes = blockToBytes(encryptBlock(feedback, key));
        const std::size_t bytesToProcess = std::min<std::size_t>(8,
data.size() - offset);
        std::array<Byte, 8> nextFeedbackBytes{};

```

```

        for (std::size_t byteIndex = 0; byteIndex < bytesToProcess;
++byteIndex)
        {
            result[offset + byteIndex] = data[offset + byteIndex] ^
gammaBytes[byteIndex];
            nextFeedbackBytes[byteIndex] = decrypting ? data[offset +
byteIndex] : result[offset + byteIndex];
        }
        feedback = bytesToBlock(nextFeedbackBytes.data());
    }
    return result;
}

std::array<Byte, 4> calculateImito(const ByteVector& data, const GKey&
key)
{
    ByteVector paddedData = data;
    if (paddedData.empty() || paddedData.size() % 8 != 0)
paddedData.resize((paddedData.size() + 7) / 8 * 8, 0);
    GBlock64 state{ 0, 0 };
    for (std::size_t offset = 0; offset < paddedData.size(); offset += 8)
    {
        const GBlock64 messageBlock = bytesToBlock(paddedData.data() +
offset);
        state.n1 ^= messageBlock.n1;
        state.n2 ^= messageBlock.n2;
        state = encryptBlock(state, key);
    }
    const auto stateBytes = blockToBytes(state);
    return { stateBytes[0], stateBytes[1], stateBytes[2], stateBytes[3]
};
}

GKey parseKey(const std::string& keyText)
{
    if (keyText.size() != 64) throw std::invalid_argument("Key must
contain 64 hexadecimal characters");
    GKey key{};
    for (std::size_t wordIndex = 0; wordIndex < key.size(); ++wordIndex)
    {
        std::uint32_t word = 0;
        for (std::size_t characterIndex = 0; characterIndex < 8;
++characterIndex)
        {
            const char character = keyText[wordIndex * 8 + characterIndex];
            word <<= 4;
            if (character >= '0' && character <= '9') word |=
static_cast<std::uint32_t>(character - '0');
            else if (character >= 'a' && character <= 'f') word |=
static_cast<std::uint32_t>(character - 'a' + 10);
            else if (character >= 'A' && character <= 'F') word |=
static_cast<std::uint32_t>(character - 'A' + 10);
            else throw std::invalid_argument("Key contains an invalid
character");
        }
        key[wordIndex] = word;
    }
    return key;
}

GBlock64 parseBlock(const std::string& blockText)
{

```

```

        if (blockText.size() != 16) throw
std::invalid_argument("Initialization vector must contain 16 hexadecimal
characters");
        std::array<Byte, 8> bytes{};
        for (std::size_t byteIndex = 0; byteIndex < bytes.size(); ++byteIndex)
bytes[byteIndex] = static_cast<Byte>(std::stoul(blockText.substr(byteIndex *
2, 2), nullptr, 16));
        return bytesToBlock(bytes.data());
    }

    std::string toHex(const std::array<Byte, 4>& bytes)
    {
        std::ostringstream result;
        result << std::hex << std::setfill('0');
        for (Byte byte : bytes) result << std::setw(2) <<
static_cast<unsigned>(byte);
        return result.str();
    }

    void printUsage(const char* programName)
    {
        std::cerr << "Usage:\n"
            << programName << " encrypt simple <input> <output>
<key64>\n"
            << programName << " decrypt simple <input> <output>
<key64>\n"
            << programName << " encrypt gamma <input> <output> <key64>
<iv16>\n"
            << programName << " decrypt gamma <input> <output> <key64>
<iv16>\n"
            << programName << " encrypt feedback <input> <output>
<key64> <iv16>\n"
            << programName << " decrypt feedback <input> <output>
<key64> <iv16>\n"
            << programName << " imito <input> <key64>\n";
    }

    int main(int argumentCount, char* argumentValues[])
    {
        try
        {
            if (argumentCount < 2) { printUsage(argumentValues[0]); return 1;
        }

        const std::string operation = argumentValues[1];
        if (operation == "imito")
        {
            if (argumentCount != 4) { printUsage(argumentValues[0]);
return 1; }

            std::cout <<
toHex(calculateImito(readBinaryFile(argumentValues[2]),
parseKey(argumentValues[3]))) << '\n';
            return 0;
        }

        if (argumentCount < 6) { printUsage(argumentValues[0]); return 1;
    }

        const std::string mode = argumentValues[2];
        const GKey key = parseKey(argumentValues[5]);
        const ByteVector inputData = readBinaryFile(argumentValues[3]);
        ByteVector outputData;
        if (mode == "simple")
        {
            if (operation == "encrypt") outputData =
encryptBlocks(inputData, key);

```

```

        else if (operation == "decrypt") outputData =
decryptBlocks(inputData, key);
        else throw std::invalid_argument("Operation must be encrypt
or decrypt");
    }
    else
    {
        if (argumentCount != 7) { printUsage(argumentValues[0]);
return 1; }
        const GBlock64 synchronizationValue =
parseBlock(argumentValues[6]);
        if (operation != "encrypt" && operation != "decrypt") throw
std::invalid_argument("Operation must be encrypt or decrypt");
        if (mode == "gamma") outputData = gammaCrypt(inputData, key,
synchronizationValue);
        else if (mode == "feedback") outputData =
feedbackCrypt(inputData, key, synchronizationValue, operation == "decrypt");
        else throw std::invalid_argument("Unknown operation mode");
    }
    writeBinaryFile(argumentValues[4], outputData);
    return 0;
}
catch (const std::exception& error)
{
    std::cerr << "Error: " << error.what() << '\n';
    return 1;
}
}

```